به نام خدا



دانشکده مهندسی کامپیوتر

درس سیستم عامل
نیم‌سال اول ۰۵-۰۶
استاد: دکتر اسدی

تمرین سری اول

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید. به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- توجه شود که پروژه نهایی درس در گروه‌های چهار نفر تحویل گرفته می‌شود.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. فرض کنید پردازنده‌ای در اختیار داریم که فاقد قابلیت پشتیبانی سخت‌افزاری از مجازی‌سازی[1] است و تنها دو سطح دسترسی حالت کاربر و حالت هسته را ارائه می‌دهد. هدف، اجرای همزمان چند سیستم‌عامل مهمان[2] روی این سخت‌افزار است.

آ– فرض کنید چند سیستم‌عامل مهمان بدون وجود لایه مجازی‌سازی و به صورت مستقیم روی سخت‌افزار اجرا شوند. توضیح دهید چرا این روش نمی‌تواند محیطی امن و قابل اعتماد برای اجرای همزمان چند سیستم‌عامل فراهم کند. سیستم‌عامل مهمان هنگام اجرا چه انتظاری از دسترسی به پردازنده، حافظه و تجهیزات ورودی/خروجی دارد؟ اگر چند سیستم‌عامل به صورت همزمان بخواهند کنترل مستقیم منابع سخت‌افزاری را در اختیار داشته باشند، چه مشکلاتی در مدیریت منابع و اجرای صحیح سیستم ایجاد خواهد شد؟ چرا وجود یک لایه واسط میان سیستم‌عامل‌های مهمان و سخت‌افزار ضروری است؟ این لایه چه وظایفی برای ایجاد جداسازی میان سیستم‌عامل‌های مختلف بر عهده دارد؟

ب– فرض کنید نرم‌افزار ناظر ماشین مجازی[3] اجازه دهد هسته[4] سیستم‌عامل مهمان مستقیماً روی سطح دسترسی هسته پردازنده واقعی اجرا شود. تحلیل کنید این طراحی چه مشکلاتی ایجاد خواهد کرد. در پاسخ خود، امنیت سیستم و جلوگیری از دسترسی غیرمجاز یک سیستم‌عامل مهمان به منابع سایر سیستم‌ها، جداسازی ماشین‌های مجازی و جلوگیری از تأثیرگذاری یک سیستم‌عامل مهمان بر عملکرد سایر سیستم‌ها، نحوه کنترل و تخصیص منابع سخت‌افزاری مانند پردازنده، حافظه و تجهیزات ورودی/خروجی و تأثیر خطاهای ایجادشده در یک سیستم‌عامل مهمان بر پایداری کل سیستم را بررسی کنید.

ج– یکی از روش‌های اولیه برای پیاده‌سازی مجازی‌سازی، روش «تله و شبیه‌سازی»[5] است. در این روش، سیستم‌عامل مهمان اجرا می‌شود و هرگاه دستوری نیازمند دسترسی ویژه باشد، اجرای آن متوقف شده و کنترل به نرم‌افزار ناظر ماشین مجازی منتقل می‌شود. توضیح دهید این روش چگونه کار می‌کند. مهم‌ترین محدودیت‌های این روش در معماری‌هایی که برای مجازی‌سازی طراحی نشده‌اند چیست؟ چرا تعداد زیاد انتقال کنترل بین سیستم‌عامل مهمان و نرم‌افزار ناظر ماشین مجازی می‌تواند باعث کاهش کارایی شود؟

د– فرض کنید معماری پردازنده نتواند تمام رفتارهای حساس سیستم‌عامل مهمان را شناسایی کرده و برای نرم‌افزار ناظر ماشین مجازی تله[6] ایجاد کند. چرا این مسئله باعث ایجاد مشکل در پیاده‌سازی مجازی‌سازی می‌شود؟ چگونه این مسئله می‌تواند جداسازی میان ماشین‌های مجازی را نقض کند؟ چرا قابلیت تشخیص صحیح عملیات حساس یکی از نیازمندی‌های اصلی یک معماری مناسب برای مجازی‌سازی است؟

ه– در پردازنده‌های جدید، قابلیت پشتیبانی سخت‌افزاری از مجازی‌سازی اضافه شده است. این قابلیت چه تغییری در طراحی و عملکرد نرم‌افزار ناظر ماشین مجازی ایجاد می‌کند؟ چرا این قابلیت باعث ساده‌تر شدن طراحی نرم‌افزار ناظر ماشین مجازی می‌شود؟

۲. وقفه[7] سازوکار اصلی سیستم‌عامل برای پاسخ‌دادن به رویدادهاست، اما همین سازوکار را می‌توان برضد سیستم به کار گرفت. در این سوال دو حمله‌ی واقعی را بررسی می‌کنیم.

بخش اول— در ماشین‌های مجازی محرمانه[8]، پردازه به ناظر ماشین مجازی[9] اعتماد ندارد، ولی تحویل وقفه‌ها هنوز دست

[1] Hardware Virtualization
[2] Guest OS
[3] Hypervisor
[4] Kernel
[5] Trap-and-Emulate
[6] Trap
[7] Interrupt
[8] Confidential VM
[9] Hypervisor

همین ناظر است. در حمله‌ی Heckler مهاجم شماره‌ی وقفه‌ای را به پردازه تزریق می‌کند که معمولاً فقط با اجرای عمدی دستور تله[۱۰] برای یک فراخوانی سیستمی ساخته می‌شود. هسته[۱۱] آن را مثل یک فراخوانی سیستمی واقعی اجرا می‌کند و با همین ترفند مرحله‌ی ورود (احراز هویت[۱۲]) در sudo و OpenSSH دور زده می‌شود.

آ- تله و وقفه در سخت‌افزار از یک جدول بردار[۱۳] و یک مسیر مشترک استفاده می‌کنند. چرا گرداننده‌ای[۱۴] که برای فراخوانی سیستمی نوشته شده، اگر به‌صورت وقفه و در لحظه‌ای ناخواسته اجرا شود خطرناک می‌شود؟

ب- هسته آرگومان‌های فراخوانی را از ثبات‌ها[۱۵] می‌خواند و نتیجه را هم در یک ثبات می‌گذارد. با این حساب چرا مقدار ثبات‌ها در لحظه‌ی حمله اهمیت دارد؟

ج- چه ویژگی‌ای یک روال رسیدگی را امن می‌کند تا اجرای آن در هر لحظه بی‌خطر باشد؟ یک راه دفاع پیشنهاد بدهید.

بخش دوم - یک پژوهش نشان داده که یک زبانه‌ی[۱۶] مرورگر، فقط با زیادکردن یک شمارنده در یک حلقه و ذخیره‌ی مرتب مقدار آن، با احتمال زیادی می‌تواند بفهمد کاربر در زبانه‌ی دیگر کدام وب‌گاه را باز کرده است.

د- دو زبانه هیچ حافظه‌ی مشترکی ندارند. پس زبانه چطور می‌فهمد در زبانه‌ی دیگر چه می‌گذرد، و چرا هر وب‌گاه الگوی خاص خودش را می‌سازد؟

ه- این سه روش را بسنجید و بگویید هر کدام تا چه میزان جلوی حمله را می‌گیرد:

۱) فرستادن وقفه‌های دستگاه به یک هسته‌ی پردازنده‌ی[۱۷] دیگر

۲) کم‌دقت و نوفه‌دار[۱۸] کردن ساعتی که در اختیار برنامه است

۳) تزریق وقفه‌های ساختگی و تصادفی

همچنین توضیح بدهید چرا وقفه‌ی زمان‌سنج[۱۹] و وقفه‌های بین‌هسته‌ای[۲۰] را نمی‌توان به هسته‌ی پردازنده‌ی دیگر منتقل کرد.

۳. در یک سرور، دو مسیر جداگانه وجود دارد: مسیر اصلی که رایانه را راه‌اندازی کرده و سیستم‌عامل را اجرا می‌کند، و کنترل‌گر مدیریت برد[۲۱] که در اسلایدها به‌صورت رایانه‌ای کوچک روی برد اصلی معرفی شده است. برای این مسئله فرض کنید کنترل‌گر مدیریت برد رایانه و مسیر مدیریتی جداگانه‌ای از میزبان دارد. حتی اگر سیستم‌عامل میزبان اجرا نشده باشد، کنترل‌گر می‌تواند وضعیت روشن یا خاموش بودن میزبان را نشان دهد و درخواست بازنشانی آن را ثبت کند. همچنین فرض کنید فرایند راه‌اندازی میزبان از میان‌افزار آغاز می‌شود، هسته[۲۲] را بارگذاری می‌کند و سپس اجرای نخستین پردازه را آغاز می‌کند.

اکنون میزبان به محیطی که اجرای پردازه‌های کاربر در آن ممکن است نمی‌رسد، اما رابط مدیریتی کنترل‌گر مدیریت برد پاسخ‌گو است.

آ- مسیر راه‌اندازی میزبان را از روشن‌شدن تا آغاز نخستین پردازه رسم کنید. کنترل‌گر مدیریت برد را در جای مناسب در همان مسیر یا در مسیری جداگانه نشان دهید.

[10] Trap
[11] Kernel
[12] Authentication
[13] Interrupt Vector Table
[14] Handler
[15] Register
[16] Tab
[17] CPU Core
[18] Noise
[19] Timer Interrupt
[20] Inter-Processor Interrupt
[21] Baseboard Management Controller (BMC)
[22] Kernel

ب- آیا رابط مدیریتی کنترل‌گر مدیریت برد را باید پردازه‌ای در سیستم‌عامل میزبان دانست؟ با توجه به صورت مسئله و ماهیت این کنترل‌گر توضیح دهید.

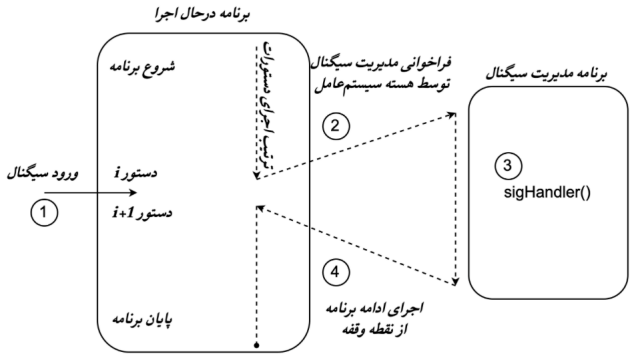
ج- آیا پاسخ‌گو بودن کنترل‌گر درحالی‌که میزبان به محیط اجرای پردازه‌های کاربر نرسیده است تناقضی ایجاد می‌کند؟ دقیقاً چه چیزی را می‌توان و چه چیزی را نمی‌توان از این مشاهده نتیجه گرفت؟

د- اگر کنترل‌گر درخواست بازنشانی میزبان را ثبت کند، پس از روشن‌شدن دوباره کدام مؤلفه هسته را بارگذاری می‌کند؟ توضیح دهید چرا ثبت درخواست بازنشانی به معنای اجرای سیستم‌عامل میزبان توسط کنترل‌گر نیست.

ه- حالتی را بررسی کنید که میزبان به درخواست‌های یک پردازهٔ کاربر پاسخ می‌دهد، اما رابط مدیریتی کنترل‌گر در دسترس نیست. آیا می‌توان نتیجه گرفت که سیستم‌عامل میزبان از کار افتاده است؟ چرا؟

و- برای دو حالت «کنترل‌گر در دسترس، سیستم‌عامل میزبان در دسترس نیست» و «کنترل‌گر در دسترس نیست، سیستم‌عامل میزبان در دسترس است»، مشخص کنید کدام مسیر کار می‌کند و کدام مسیر ممکن است از کار افتاده باشد.

۴. سیگنال‌ها، اعلان‌های ورودی به برنامه هستند. از آنجایی که عموماً منشأ نرم‌افزاری دارند گاهی به آن‌ها وقفه نرم‌افزاری[۲۳] یا تله[۲۴] نیز گفته می‌شود. سیگنال‌ها روند عادی برنامه را متوقف می‌کنند و در اغلب مواقع پیش‌بینی زمان ورود آن‌ها ناممکن است. منشأ اصلی صدور سیگنال‌ها از هسته سیستم است اما امکان ارسال و دریافت سیگنال بین دو برنامه درحال اجرا نیز وجود دارد که از آن می‌توان برای همگام‌سازی و ارتباطات بین فرآیندی[۲۵] استفاده کرد که در فصل‌های بعدی با این مفاهیم به تفصیل آشنا خواهید شد.



شکل ۱: نمایی از ساختار سیگنال‌ها در سیستم‌عامل

(آ) پس از ورود یک سیگنال، ممکن است فرآیند به صورت پیش‌فرض مجموعه‌ای از رفتارها را در بازخورد به آن سیگنال انجام دهد. تعداد این رفتارهای پیش‌فرض محدود است، آن‌ها را بررسی کنید و دو نمونه از آن‌ها را شرح دهید.

(ب) در لینوکس تقریباً تمامی ابزارها و اجزا به شکل فایل‌های مجزا هستند. یکی از این انواع فایل‌ها در مسیر زیر قرار دارد. سه مؤلفه موجود در این فایل که درباره سیگنال‌های یک فرآیند هستند را مشخص کرده و کارکرد آن‌ها را بیان کنید.

```
/proc/PID/status (e.g. /proc/10/status)
```

(ج) یکی از امکانات سیستم‌عامل لینوکس قابلیت مدیریت این سیگنال‌ها (Deposition Signals) و ارائه روندی از پیش تعیین شده در مواقع بروز این نوع وقفه‌ها است. هر سیگنال دارای نام و عددی است که دلیل صدور آن را

[23] Software Interrupts
[24] Trap
[25] Inter-Process Communication

نشان می‌دهد. بررسی کنید برای شناسایی تله ناشی از فراخوانی سامانه‌ای نامعتبر و خطا در محاسبات شناور چه سیگنال‌هایی صادر می‌شوند.

(د) در سیستم‌عامل امکان mask یک وقفه وجود دارد. چرا لازم است چنین ویژگی در سیستم‌عامل وجود داشته باشد؟

تمارین عملی

۱. تمرینات عملی این درس مبتنی بر سیستم‌عامل xv6 است که توسط دانشگاه MIT برای اهداف آموزشی ساخته شده است. لینک مستندات و فیلم‌های آموزشی مربوط به این سیستم‌عامل در صفحه cw درس قرار داده شده است.

۱. با استفاده از فیلم آموزش موجود در cw، سیستم‌عامل xv6 را اجرا کنید و از مراحل آن اسکرین‌شات تهیه کنید.
پیشنهاد ما این است که نسخه 13 توزیع Debian و یا 24 Ubuntu استفاده کنید. در صورتی که از سیستم‌عامل لینوکس استفاده نمی‌کنید، می‌توانید به کمک لینک‌های زیر یک ماشین مجازی Debian و یا Ubuntu را بیاورید:
https://www.makeuseof.com/how-to-install-debian-on-virtualbox/
https://www.geeksforgeeks.org/linux-unix/how-to-install-ubuntu-on-virtualbox/
پیش از شروع، مطابق همان فیلم آموزش، پیش‌نیازهای لازم برای ساخت و اجرای xv6 (ابزارهای کامپایل و QEMU) را نیز نصب کنید.
لطفاً توجه کنید که حداقل نسخه مورد نیاز Qemu برای اجرای xv6، نسخه 7.2 است. اگر قصد استفاده از توزیع‌های دیگر لینوکس را دارید، حتماً به این نکته توجه کنید.

۲. در این بخش قصد داریم تا یک برنامه کاربر به سیستم‌عامل اضافه کنیم تا با اجرای آن عبارت !Hello World در خروجی ترمینال چاپ شود. بدین منظور نیاز است تا تغییرات را در بخش کاربر سیستم‌عامل اعمال کنید و همچنین این برنامه جدید را به Makefile نیز اضافه کنید. در نهایت این برنامه کاربر را اجرا کنید و از خروجی آن اسکرین‌شات تهیه کنید.

۳. در این بخش قصد داریم یک System Call سفارشی به سیستم‌عامل اضافه کنیم. نام این System Call برابر power است که دو عدد صحیح a و b را به عنوان ورودی می‌گیرد و مقدار a به توان b را به فضای کاربر برمی‌گرداند. سپس برنامه کاربری‌ای بنویسید که این System Call را فراخوانی کرده و نتیجه را چاپ کند. خروجی مورد انتظار برای برنامه شما چیزی شبیه به زیر است:

```
$ power
2^10 = 1024
3^4 = 81
```

پیاده‌سازی شما باید هر دو بخش زیر را کامل کند:

- **سمت هسته:** تعریف شماره System Call، تابع اجرای آن در هسته و ثبت آن در جدول فراخوانی
- **سمت کاربر:** تعریف System Call جدید در فضای کاربر و نوشتن برنامه‌ای که آن را فراخوانی می‌کند

نکته: برای سادگی، فرض کنید a و b اعدادی غیرمنفی هستند و نتیجه در محدوده یک عدد صحیح 32 بیتی جا می‌شود.
در نهایت برنامه را کامپایل کنید، چند بار با ورودی‌های مختلف آن را در xv6 اجرا کنید و از خروجی آن اسکرین‌شات تهیه کنید.

خروجی‌های قابل تحویل

۱. یک آرشیو فشرده (zip.) شامل:
- کد منبع برنامه Hello World و فایل تغییر داده Makefile
- کد منبع برنامه power و فایل‌های تغییر داده‌شده برای پیاده‌سازی System Call
- اسکرین‌شات‌های اجرای xv6 و برنامه‌ها

۲. یک ویدیوی حداکثر 6-5 دقیقه‌ای که در آن موارد زیر را نمایش دهید:
- اجرای xv6 و اجرای چند دستور ساده در آن
- اجرای برنامه Hello World

- اجرای برنامه power (حداقل دو بار با ورودی‌های مختلف)
- توضیح کوتاه مسیر رفتن فراخوانی System Call از فضای کاربر به هسته و بازگشت آن به فضای کاربر

در ویدیو مراحل انجام‌شده را به صورت کوتاه توضیح دهید و نیازی به توضیح خط‌به‌خط کدها نیست.

۲. در این بخش با نصب داکر، دریافت Image، ساخت Container و مدیریت چرخهٔ عمر کانتینرها آشنا می‌شوید.

۱. **نصب (۱۰ امتیاز):** Docker Engine را روی یک میزبان لینوکسی نصب کنید. نسخهٔ داکر و خروجی اجرای موفق hello-world را ثبت کنید. از دستورهای زیر استفاده کنید:

```
docker --version
docker info
docker run --rm hello-world
```

۲. **تصویر و کانتینر (۱۵ امتیاز):** تصویر ubuntu:24.04 را دریافت کنید، فهرست تصویرها را ببینید و کانتینری تعاملی با نام os اجرا کنید. درون آن، نسخهٔ هسته و اطلاعات توزیع را مشاهده کنید، یک فایل در مسیر /tmp/os.txt بسازید و خارج شوید.

```
docker pull ubuntu:24.04
docker images
docker run -it --name os ubuntu:24.04 bash
uname -r
cat /etc/os-release
touch /tmp/os.txt
exit
```

۳. **مدیریت کانتینر (۱۰ امتیاز):** با docker ps -a وضعیت os را بررسی کنید؛ آن را با docker start -ai os دوباره اجرا کنید و باقی ماندن فایل را بسنجید. سپس کانتینری با دستور زیر در پس‌زمینه اجرا کنید؛ وضعیت آن را ببینید و به ترتیب متوقف، راه‌اندازی مجدد و حذف کنید. در پایان os را نیز حذف کنید.

```
docker run -d --name os-bg ubuntu:24.04 sleep 600
docker ps
docker stop os-bg
docker start os-bg
docker stop os-bg
docker rm os-bg
docker rm os
```

سؤال (۵ امتیاز): تفاوت Image و Container چیست؟ چرا خروج از شِل کانتینر را متوقف می‌کند ولی فایل ساخته‌شده پس از راه‌اندازی مجدد باقی می‌ماند؟ (حداکثر پنج خط)

مقایسهٔ Bare Metal، ماشین مجازی و داکر

در این بخش قصد داریم ساختار و کارایی سه محیط اجرای Bare Metal، ماشین مجازی (VM) و داکر را مقایسه کنیم. برای این بخش به یک **میزبان لینوکسی با نصب مستقیم روی سخت‌افزار** نیاز دارید؛ داکر را روی همین میزبان و یک ماشین مجازی لینوکسی را نیز روی آن راه‌اندازی کنید. ابزار sysbench را در هر سه محیط نصب کنید. **داکر دسکتاپ در ویندوز یا مک جایگزین میزبان Bare Metal نیست.**

توضیح تمرین: سه محک زیر را در هر محیط اجرا و نتایج را مقایسه کنید:

محک cpu-intensive:

```
sysbench cpu --threads=4 --cpu-max-prime=2000000 --percentile=99 run
```

محک memory-intensive:

```
sysbench memory --memory-total-size=20G --memory-access-mode=rnd run
```

محک io-intensive:

```
sysbench fileio --file-total-size=4G prepare
sysbench fileio --file-total-size=4G --file-test-mode=seqwr --percentile=99 run
sysbench fileio --file-total-size=4G cleanup
```

سؤال (۱۰ امتیاز): عملکرد هر محک را در یک خط توضیح دهید. پیش از اجرا، کارایی نسبی سه محیط را برای هر محک پیش‌بینی و دلیل خود را به اختصار بیان کنید.

هر محک را در هر سه محیط اجرا کنید و **سه نمودار** (برای هر محک یک نمودار با سنجه‌های مشخص زیر) بسازید:

- cpu-intensive: P99 Latency Average Latency
- memory-intensive: Total Time Throughput
- io-intensive: P99 Latency Average Latency Write Throughput

نکته: برای مقایسهٔ منصفانه، نسخهٔ sysbench و پارامترهای آزمون یکسان باشند؛ تعداد vCPU و حافظهٔ ماشین مجازی و روش دسترسی به فایل‌ها در داکر (bind mount یا فایل‌سیستم کانتینر) را در گزارش ذکر کنید. هر آزمون را دست‌کم سه بار تکرار و مقدار میانه را رسم کنید. اگر محک پردازنده تعداد رویداد کافی برای سنجش صدک تولید نکرد، مقدار cpu-max-prime را **به‌طور یکسان در هر سه محیط** کاهش دهید و تغییر را ثبت کنید. برای سنجه‌های با واحد متفاوت، در نمودار از زیرنمودارهای جدا استفاده کنید.

سؤال (۱۰ امتیاز): آیا نتایج با پیش‌بینی شما یکسان بود؟ تفاوت‌های مشاهده‌شده را با توجه به نحوهٔ اجرای پردازه‌ها، دسترسی به هسته و سربار مجازی‌سازی در هر محیط به اختصار توجیه کنید.

موارد لازم جهت تحویل: یک فایل PDF شامل دستورهای مهم و خروجی اجرای آن‌ها، پاسخ پرسش‌ها، **سه نمودار**، مشخصات مختصر محیط‌های اجرا و توضیح کوتاهی از مراحل کار. امتیاز بخش مقایسه: اجرا و نمودارها (۴۰)، پیش‌بینی و توضیح محک‌ها (۱۰)، تحلیل نتایج (۱۰).

۳. هدف از این سوال، آشنایی عملی با فرآیند کامپایل لینوکس و افزودن ماژول و فراخوانی سیستم[۲۶] سفارشی برای اهداف توسعه و دیباگ است. در این تمرین، به طور خلاصه باید کارهای زیر را انجام دهید:

- کد منبع هسته لینوکس را دانلود و پیکردبندی کنید.
- ماژول‌های خاص مورد نیاز برای دیباگ را به پیکربندی هسته اضافه کنید.
- هسته را با پیکربندی جدید کامپایل کنید.
- یک ماژول ساده هسته (.ko) بنویسید و کامپایل کنید.
- ماژول کامپایل شده خود را به هسته اضافه و حذف کنید.

پیش‌نیاز

- یک سیستم مجازی یا فیزیکی با یک توزیع لینوکس (ترجیحا دبیان ۱۲)
- **توجه :** نام کاربری این سیستم‌عامل را، نام خود و هم‌گروهی‌تان قرار دهید

osvm@lastname1-lastname2$

- حداقل ۱۵ گیگابایت فضای آزاد دیسک

[26] System Call

- ابزارهای لازم برای کامپایل کرنل :
بعد از نصب سیستم‌عامل این دستورات را اجرا کنید :

```
sudo apt update

sudo apt install -y \
    build-essential \
    bc \
    bison \
    flex \
    libssl-dev \
    libelf-dev \
    libncurses-dev \
    dwarves \
    fakeroot \
    cpio \
    rsync \
    kmod
```

گام‌های تمرین

۱. **دریافت کد منبع هسته**

(آ) ابتدا کد منبع هسته ورژن 6.1.189 را از اینجا دریافت کنید.

۲. **پیکربندی هسته**

(آ) از فایل پیکربندی فعلی سیستم خود کپی بگیرید تا یک نسخه پشتیبان داشته باشید.

```
cp /boot/config-\$(uname -r) .config
```

(ب) منوی پیکربندی هسته را با دستور make menuconfig اجرا کنید.

(ج) در منوی پیکربندی موارد، ماژول‌های زیر را فعال کنید.

- Kernel Hacking → Kernel debugging
- Kernel Hacking → Memory debugging(گزینه‌هایی مانند KASAN را فعال کنید)
- Enable loadable module support(در صورت غیرفال بودن)

(د) **نکته:** با استفاده از دستور make localmodconfig می‌توانید با حذف درایورهای غیرضروری از build، سرعت کامپایل را افزایش دهید.

۳. **کامپایل هسته**

(آ) با استفاده از دستورات زیر، هسته و ماژول‌ها را کامپایل کنید.(این مرحله ممکن است مدت زیادی طول بکشد)

```
make -j$(nproc)
sudo make modules_install
sudo make install
```

(ب) GRUB را بروزسانی کنید و سیستم را reset کنید و هسته جدید را انتخاب کنید.

(ج) با اجرای دستور زیر از تغییر نسخه هسته مطمئن شوید.

```
uname -r
```

۴. **توسعه و تست یک ماژول ساده**

(آ) یک ماژول ساده هسته ایجاد کنید که هنگام بارگذازی و حذف شدن، پیامی را در /var/log/kern.log چاپ کند.

(ب) یک Makefile ساده برای کامپایل ماژول بنویسید.
(ج) ماژول را با استفاده از make کامپایل کنید.
(د) دستوراتی که باعث بارگذاری و حذف یک ماژول در در فضای هسته می‌شود را پیدا کرده و با اجرای آن‌ها، ماژول خود را در هسته بارگذاری و حذف کنید.
(ه) خروجی لاگ را با دستور زیر و یا با بررسی فایل /var/log/kern.log مشاهده کنید.

```
sudo dmesg | tail
```

۵. افزودن و استفاده از یک System Call

(آ) یک System Call با نام hello_syscall به هسته اضافه کنید. این System Call باید بدون دریافت ورودی، مقدار 42 را برگرداند و پیام زیر را در لاگ هسته چاپ کند:

```
hello_syscall: Hello from the kernel!
```

(ب) System Call خود را در محل مناسب کد هسته پیاده‌سازی کرده و آن را در جدول System Callهای معماری x86-64 ثبت کنید.
(ج) هسته را مجدداً کامپایل و نصب کرده و سیستم را با هسته جدید راه‌اندازی کنید.
(د) یک برنامه ساده با زبان C بنویسید که System Call جدید را از فضای کاربر فراخوانی کند و مقدار بازگشتی آن را چاپ کند. برای مثال:

```
Calling hello_syscall...
hello_syscall returned: 42
```

(ه) برنامه را کامپایل و اجرا کنید. سپس با استفاده از دستور زیر پیام چاپ‌شده توسط System Call را مشاهده کنید:

```
sudo dmesg | tail
```

(و) در ویدیوی خود توضیح دهید هنگام اجرای System Call چگونه اجرای برنامه از فضای کاربر به فضای هسته منتقل شده و پس از اجرای System Call به فضای کاربر بازمی‌گردد.

خروجی‌های قابل تحویل

لطفاً موارد زیر را ارسال کنید:

۱. یک آرشیو فشرده (zip.) شامل:
- فایل پیکربندی نهایی هسته (config.)
- کد منبع ماژول (c.)
- Makefile مربوط به ماژول
- کد منبع برنامه فضای کاربر برای اجرای System Call
- فایل‌های تغییر داده‌شده برای پیاده‌سازی System Call

۲. یک ویدیوی حداکثر 6-5 دقیقه‌ای که در آن موارد زیر را نمایش دهید:
- اجرای هسته کامپایل‌شده
- کامپایل، بارگذاری و حذف ماژول و پیام‌های آن در لاگ
- تغییرات مربوط به System Call
- اجرای برنامه فضای کاربر و فراخوانی موفق System Call
- مقدار بازگشتی و پیام System Call در لاگ هسته

در ویدیو مراحل انجام‌شده را به صورت کوتاه توضیح دهید و نیازی به توضیح خط‌به‌خط کدها نیست.